EDA

Exploratory Data Analysis (EDA) is the foundational phase of data science where you analyze datasets to summarize their primary characteristics, uncover underlying structures, spot anomalies, test hypotheses, and check assumptions before formal modeling or dashboarding.

Here is a comprehensive breakdown of EDA, real-world data sources, and a complete roadmap to build your interactive Streamlit dashboard.

1. Everything You Need to Know About EDA

EDA is structured around four primary analysis paradigms:

A. Core Techniques

- **Univariate Analysis:** Analyzing single variables in isolation.
  - *Numerical:* Histograms, Box Plots, Density Curves (KDE) to inspect mean, median, skewness, variance, and extreme values.
  - *Categorical:* Frequency tables and Bar Charts to assess category distributions and class imbalances.
- **Bivariate Analysis:** Examining relationships between two variables.
  - *Numeric vs. Numeric:* Scatter plots, Pearson/Spearman correlation matrices.
  - *Numeric vs. Categorical:* Grouped box plots, violin plots, grouped aggregations (`groupby`).
  - *Categorical vs. Categorical:* Cross-tabulations (`crosstab`), stacked bar charts, Chi-Square test of independence.
- **Multivariate Analysis:** Analyzing interactions across three or more variables simultaneously (e.g., Pairplots, Heatmaps, multi-dimensional scatter plots using color, size, and facets).

B. Critical EDA Checklist

1. **Structural Inspection:** Shape (`rows × columns`), data types (`dtypes`), memory usage, and first/last records.

2. **Missingness Diagnosis:** Determine if missing values are Missing Completely at Random ($MCAR$), Missing at Random (MAR), or Missing Not at Random ($MNAR$).
3. **Outlier Detection:** Use Interquartile Range ($IQR = Q3 - Q1$) bounds or Z-scores ($\vert{}Z\vert{} > 3$) to separate measurement noise from genuine tail behavior.
4. **Data Integrity & Leakage:** Identify duplicates, invalid ranges (e.g., negative prices), inconsistent text casing/whitespace, and feature leakage (information present in training data that wouldn't be available in real-time).

2. Where to Get High-Quality Real-World Datasets

To make your project portfolio-ready, use raw, untidy datasets that require actual transformation:

Dataset Domain	Source / Link	What Makes It Ideal for EDA & Dashboards
Airbnb Listings	Inside Airbnb	Uncleaned, rich geo-spatial data, pricing, host metrics, and text reviews across major global cities.
E-Commerce Transactions	UCI Machine Learning Repository	Real transaction data with negative quantities (returns), missing customer IDs, multi-currency values, and temporal trends.
Financial Markets / Macro	Yahoo Finance API (`yfinance`) / FRED	Live/historical OHLCV stock data, macroeconomic indicators, and time-series volatility requiring real-time parsing.
Public Datasets	Kaggle Datasets / Google Dataset Search	Broad repository for niche real-world datasets across industries.

3. End-to-End Project Execution Roadmap

To build a modular, production-ready Streamlit app, structure your project with clean separation of concerns:

Plaintext

```
ecommerce_eda_dashboard/
|
```

```
├── data/
│   ├── raw_sales.csv            # Original raw file
│   └── cleaned_sales.parquet    # Processed output (fast read speed)
│
├── src/
│   ├── data_loader.py           # Ingestion & cleaning functions
│   └── visualizations.py       # Plotly chart builders
│
├── app.py                       # Main Streamlit web application
├── requirements.txt             # Dependencies
└── README.md                    # Documentation & insights
```

Step 1: Ingestion & Data Transformation (`src/data_loader.py`)

Handle missing values, parsing, and type conversions cleanly with Pandas:

Python

```
import pandas as pd
import numpy as np

def clean_ecommerce_data(file_path: str) -> pd.DataFrame:
    # Load dataset
    df = pd.read_csv(file_path, encoding="ISO-8859-1")

    # 1. Drop complete duplicates
    df.drop_duplicates(inplace=True)

    # 2. Handle invalid records (e.g., returns or cancellations)
    df = df[(df["Quantity"] > 0) & (df["UnitPrice"] > 0)]

    # 3. Type Conversion & Missing Value Treatment
    df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"])
    df["CustomerID"] = df["CustomerID"].fillna(-1).astype(int)
    df["Description"] = df["Description"].str.strip().str.upper()
    df["Description"].fillna("UNKNOWN ITEM", inplace=True)

    # 4. Feature Engineering
    df["TotalAmount"] = df["Quantity"] * df["UnitPrice"]
    df["YearMonth"] = df["InvoiceDate"].dt.to_period("M").astype(str)
    df["DayOfWeek"] = df["InvoiceDate"].dt.day_name()

    return df
```

Step 2: Interactive Web App (`app.py`)

Build the interface with dynamic sidebar filters and responsive Plotly visual charts:

Python

```
import streamlit as st
import plotly.express as px
from src.data_loader import clean_ecommerce_data

# Page configuration
st.set_page_config(page_title="E-Commerce EDA Dashboard", layout="wide")

st.title("📊 E-Commerce Real-Time Exploratory Analytics")

# Cache data loading to optimize performance
@st.cache_data
def load_data():
    return clean_ecommerce_data("data/raw_sales.csv")

df = load_data()

# Sidebar: Cross-filtering controls
st.sidebar.header("Global Filters")
countries = st.sidebar.multiselect(
    "Select Country:",
    options=df["Country"].unique(),
    default=["United Kingdom", "Germany", "France"]
)

# Apply filters
filtered_df = df[df["Country"].isin(countries)] if countries else df

# Metric KPIs
col1, col2, col3, col4 = st.columns(4)
col1.metric("Total Revenue", f"${filtered_df['TotalAmount'].sum():,.2f}")
col2.metric("Total Orders", f"{filtered_df['InvoiceNo'].nunique():,}")
col3.metric("Avg Order Value", f"${filtered_df.groupby('InvoiceNo')['TotalAmount'].sum
().mean():,.2f}")
col4.metric("Unique Customers", f"{filtered_df[filtered_df['CustomerID'] != -1]['Custo
merID'].nunique():,}")

st.markdown("---")

# Visual Analytics Section
tab1, tab2 = st.tabs(["📈 Revenue Trends", "📦 Top Products & Geography"])

with tab1:
    st.subheader("Monthly Revenue Trajectory")
    monthly_rev = filtered_df.groupby("YearMonth")["TotalAmount"].sum().reset_index()
    fig_line = px.line(monthly_rev, x="YearMonth", y="TotalAmount", markers=True,
                       title="Revenue over Time", labels={"TotalAmount": "Revenue
($)"})
    st.plotly_chart(fig_line, use_container_width=True)

with tab2:
    col_left, col_right = st.columns(2)
    with col_left:
        st.subheader("Top 10 Products by Sales")
        top_products = filtered_df.groupby("Description")["TotalAmount"].sum().nlarges
t(10).reset_index()
```

```
        fig_bar = px.bar(top_products, x="TotalAmount", y="Description", orientation
="h",
                         color="TotalAmount", color_continuous_scale="Viridis")
        st.plotly_chart(fig_bar, use_container_width=True)

    with col_right:
        st.subheader("Revenue by Country")
        country_rev = filtered_df.groupby("Country")["TotalAmount"].sum().reset_index
()
        fig_map = px.choropleth(country_rev, locations="Country", locationmode="countr
y names",
                                color="TotalAmount", color_continuous_scale="Blues")
        st.plotly_chart(fig_map, use_container_width=True)
```

Step 3: Deployment to Streamlit Community Cloud

1.Create requirements.txt:

Specify exact dependencies for build environment.

Create a `requirements.txt` file in your root folder listing all required packages:

Plaintext

```
streamlit
pandas
plotly
openpyxl
```

2.Push Project to GitHub:

Version control repository setup.

Initialize git, commit your files, and push the repository to GitHub:

Bash

```
git init
git add .
git commit -m "Initial commit for EDA dashboard"
git branch -M main
git push -u origin main
```

3.Deploy on Streamlit Cloud:

Connect GitHub repository for automated continuous deployment.

1. Sign in to share.streamlit.io using your GitHub account.

2. Click **New app**.
3. Select your repository, branch (`main`), and set the main file path to `app.py`.
4. Click **Deploy**. Your live web dashboard will be generated with a public URL.